TUTORIAL DE GIT · CAPÍTULO 2

Fundamentos de Git

El ciclo de trabajo básico: obtener, modificar, confirmar, deshacer y sincronizar

1. Obtener un repositorio

Hay dos formas de empezar a trabajar con un repositorio Git:

```
# a) Convertir una carpeta existente en repositorio
cd mi_proyecto
git init

# b) Clonar un repositorio remoto (descarga toda su historia)
git clone https://github.com/usuario/repo.git
git clone https://github.com/usuario/repo.git otro_nombre   # con otro nombre
de carpeta
```

`git init` crea la carpeta oculta `.git` con la estructura del repositorio. `git clone` crea la carpeta, inicializa `.git`, descarga todos los datos y deja listo el contenido de la rama principal.

2. Ciclo de vida de los archivos

Cada archivo del directorio de trabajo está **sin seguimiento** (*untracked*) o **bajo seguimiento** (*tracked*). Los archivos con seguimiento pueden estar sin modificar, modificados o preparados.

```
Sin seguimiento --git add--> Preparado
Sin modificar --editar--> Modificado --git add--> Preparado --git commit-->
Sin modificar
```

3. Registrar cambios en el repositorio

Ver el estado: git status

```
git status        # salida completa
git status -s     # salida corta: ?? sin seguimiento, A añadido, M modificado
```

Preparar archivos: git add

```
git add archivo.txt      # un archivo
git add .                # todos los cambios de la carpeta actual
git add *.txt            # todos los .txt
```

`git add` es un comando multiuso: empieza a seguir archivos nuevos, prepara archivos modificados y marca como resueltos los conflictos de una fusión.

Ignorar archivos: .gitignore

El archivo `.gitignore` lista los patrones de archivos que Git no debe seguir (compilados, claves, archivos temporales):

```
*.log          # ignora todos los .log
*.tmp
build/         # ignora la carpeta build
!importante.log   # excepción: sí se sigue este archivo
```

Ver los cambios: git diff

```
git diff            # cambios sin preparar (directorio de trabajo vs. área de preparación)
git diff --staged   # cambios preparados (área de preparación vs. último commit)
```

Confirmar los cambios: git commit

```
git commit                      # abre el editor para escribir el mensaje
git commit -m "Mensaje descriptivo"
git commit -a -m "Mensaje"      # salta el git add de los archivos ya seguidos
```

Eliminar y mover archivos

```
git rm archivo.txt            # borra el archivo y deja el borrado preparado
git rm --cached archivo.txt   # deja de seguirlo pero lo conserva en disco
git mv origen.txt destino.txt # renombra o mueve el archivo
```

4. Ver el historial de confirmaciones

```
git log                       # historial completo
git log -p -2                 # cambios (parche) de las 2 últimas confirmaciones
git log --stat                # resumen de archivos modificados
git log --oneline             # una línea por commit
git log --graph --oneline --all   # historial con gráfico de ramas
git log --pretty=format:"%h %an %ar : %s"
```

5. Deshacer cosas

Situación	Comando
Corregir el último commit (mensaje u olvidar un archivo)	`git commit --amend`
Sacar un archivo del área de preparación	`git reset HEAD archivo` o `git restore --staged archivo`
Descartar los cambios de un archivo modificado	`git checkout -- archivo` o `git restore archivo`

> *Cuidado: descartar cambios con `git restore` o `git checkout --` es irreversible, porque esos cambios nunca se confirmaron. Tampoco conviene usar `--amend` sobre commits que ya se enviaron al remoto.*

6. Trabajar con repositorios remotos

```
git remote -v                           # listar remotos con su URL
git remote add origin https://github.com/usuario/repo.git
git fetch origin                        # descarga datos sin fusionarlos
git pull                                # fetch + fusión en la rama actual
git push origin main                    # enviar los commits al remoto
git remote show origin                  # inspeccionar un remoto
git remote rename origin nuevo
git remote remove nuevo
```

Al clonar, Git nombra automáticamente `origin` al servidor de origen. `git push` solo funciona si se tiene permiso de escritura y si nadie más envió cambios antes; en ese caso hay que hacer `git pull` primero.

7. Etiquetas (tags)

Las etiquetas marcan puntos importantes de la historia, normalmente versiones (v1.0, v2.0).

```
git tag                          # listar etiquetas
git tag -a v1.0 -m "Versión 1.0"  # etiqueta anotada (guarda autor, fecha y
mensaje)
git tag v1.0-ligera              # etiqueta ligera (solo un puntero)
git push origin v1.0             # las etiquetas no se envían con push normal
git push origin --tags           # enviar todas las etiquetas
```

8. Alias

```
git config --global alias.co checkout
git config --global alias.ci commit
git config --global alias.st status
git config --global alias.last "log -1 HEAD"
```

> *Resumen: el ciclo básico es modificar, `git add`, `git commit` y `git push`. Con `git status`, `git diff` y `git log` se revisa en todo momento qué pasó.*

Fuente: Adaptado de Chacon, S. y Straub, B. (2014). Pro Git (2.ª ed.). Apress. https://git-scm.com/book/es/v2 (licencia CC BY-NC-SA 3.0).